

Reviewer Pack — Minimal Order of Automorphic Forms and DPLL Proof Width Corr...

Entry 3fc8f3f856b3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 12:45:18 UTC

Minimal Order of Automorphic Forms and DPLL Proof Width Correlation

Entry ID: 3fc8f3f856b3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 12:45:18 UTC

1. Conjecture

Field A (mathematical branch): Automorphic Form Theory **Field B** (complexity object): Complexity Theory (DPLL Proof Complexity)

Statement:

The minimal order of automorphic forms associated with a given Boolean satisfiability problem (SAT) instance φ is linearly correlated with the width of its DPLL proof tree, such that $|O(\varphi)| = \Theta(w_{\text{DPLL}}(\varphi))$, where $w_{\text{DPLL}}(\varphi)$ is the maximum depth of any path in the DPLL proof tree for instance φ .

Rationale (proposer's reasoning):

Automorphic forms are a generalization of modular forms and are known to exhibit rich algebraic structures. By linking this area with complexity theory, we might uncover new ways to characterize computational hardness. The linear correlation between the order of these forms and the DPLL proof width could provide insights into the inherent difficulty of solving SAT problems.

Taxonomy category: AutomorphicFormTheory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b2e52cab75bf2525

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between $|O(\varphi)|$ and $w_{\text{DPLL}}(\varphi)$ across 30 randomly generated SAT instances φ ($n \leq 40$) exceeds a threshold of 0.5, where $|O(\varphi)|$ represents the minimal order of automorphic forms and $w_{\text{DPLL}}(\varphi)$ is the width of the DPLL proof tree for instance φ .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - automorphic form AND DPLL proof width - SAT instance automorphic forms correlation DPLL proof complexity - linear relation minimal order automorphic forms DPLL proof tree

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=1.8s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_sat_instance(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def dpll(instance, assignment, free_vars, clause_set):
        if not clause_set:
            return True
        if not free_vars:
            return False

        p = free_vars[0]
        new_assignment = assignment[:]
        new_assignment[p] = 0
        if dpll(instance, new_assignment, free_vars[1:], clause_set):
            return True

        new_assignment[p] = 1
        if dpll(instance, new_assignment, free_vars[1:], clause_set):
            return True

        return False

    def compute_dpll_width(instance):
        n = int(math.log2(len(instance)))
        assignment = [None] * (n + 1)
```

```

    free_vars = list(range(1, n + 1))
    return dpll(instance, assignment, free_vars, clause_set)

def compute_minimal_order(instance):
    # Placeholder for the actual computation of minimal order
    # This is a dummy implementation and should be replaced with the actual algorithm
    return len(instance) / 2

n_values = [5, 10, 15, 20, 30, 40]
total_metric_value = 0
instances_tested = 0
n_max = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    for _ in range(5): # Ensure at least 30 instances per seed
        instance = generate_sat_instance(n)
        min_order = compute_minimal_order(instance)
        dpll_width = compute_dpll_width(instance)

        total_metric_value += abs(min_order - dpll_width)
        instances_tested += 1
        n_max = max(n_max, n)

        if abs(min_order - dpll_width) > 0.5:
            conjecture_holds = False
            counterexample = f"Instance of size {n} with min_order={min_order} and dpll_width="

    return {
        "metric_name": "Absolute Difference",
        "metric_value": total_metric_value / instances_tested,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{result}...}}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) >= 0.8:
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing={results[0]['n']}")

```

```

else:
    print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_acaf2b62.py", line 89, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_acaf2b62.py", line 64, in run_trial
    dpll_width = compute_dpll_width(instance)
                  ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_acaf2b62.py", line 46, in compute_dpll
    return dpll(instance, assignment, free_vars, clause_set)
           ^^^^^^^^^^^^^
NameError: name 'clause_set' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the Pearson correlation coefficient could not be calculated to verify the conjecture. | next: Re-run the test with proper error handling and ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14878
2	preregistration	ollama_remote	glm4:latest	0	0	10327
3	novelty	ollama_remote	glm4:latest	0	0	8951
4	novelty	ollama_remote	glm4:latest	0	0	9771
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16264
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9997
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16217
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16410
9	judge	ollama_remote	glm4:latest	0	0	19451

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 122267 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/3fc8f3f856b3.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/3fc8f3f856b3.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/3fc8f3f856b3.tar.gz (if generated)